

INFORME TÉCNICO

Investigación e Implementación de API Interna
API de Eficiencia de Operarios
Integración en Texticode — Sistema de Gestión Textil

Aprendiz Kevin Stevan Wagner Kevin Andres Castro Andres Felipe Lancheros Camilo Andres Tibambre	Instructor Ing. Edwin Marín
Fecha: Mayo 2026	Proyecto: Texticode — ADSO 2026

★ Implementación real verificada con evidencias de prueba ★

1. API IMPLEMENTADA: EFICIENCIA DE OPERARIOS

1.1 Justificación de la elección

La API de Eficiencia de Operarios es una API interna desarrollada completamente dentro del backend de Texticode (Node.js + Express). A diferencia de una API externa, no depende de servicios de terceros, no tiene costos de uso y accede directamente a la base de datos MySQL del sistema.

Para el proyecto Texticode — sistema de gestión de producción textil — esta API es completamente natural porque el sistema ya registra órdenes de producción, operarios asignados y estados de avance, pero carecía de un mecanismo para analizar el rendimiento del equipo de producción.

Necesidad en Texticode	Solución implementada con la API
El admin no sabía qué operario rendía mejor	Endpoint que calcula prendas producidas por día, órdenes en retraso y nivel de rendimiento por operario
No existía forma de filtrar operarios por nivel	Query params: ?rendimiento=Alto Medio Bajo filtra la lista automáticamente
Ver el detalle de un operario específico era manual	Path param: /operarios/:id devuelve perfil completo + todas sus órdenes
Acceso sin control a información sensible del	API Key en header x-api-key protege todos los

equipo	endpoints de la API
--------	---------------------

1.2 Problema que resuelve la API

Texticode registraba las órdenes y los operarios asignados, pero no tenía ningún mecanismo para medir el rendimiento del equipo. El administrador no sabía cuántas prendas producía cada operario por día ni cuáles tenían órdenes vencidas sin completar. La API resuelve tres necesidades reales:

- Ranking de eficiencia: lista todos los operarios ordenados por prendas producidas por día, con su nivel de rendimiento (Alto / Medio / Bajo) calculado en base a productividad real y retrasos.
- Filtrado dinámico: el administrador puede filtrar por rendimiento, estado de órdenes o limitar el número de resultados usando Query params.
- Perfil detallado: con un Path param el administrador ve el perfil completo de un operario específico más el detalle de todas sus órdenes asignadas.

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Descripción técnica

Tipo: API Interna REST | Framework: Express.js | BD: MySQL (Texticode) | Archivo: routes/eficiencia.js | Seguridad: API Key en header x-api-key

2.2 Métodos HTTP y endpoints

Método	Endpoint	Uso en Texticode	Respuesta
GET	/api/eficiencia/operarios	Lista todos los operarios con métricas de rendimiento	200 OK + JSON
GET	/api/eficiencia/operarios?rendimiento=Alto	Filtra solo operarios de rendimiento Alto (Query param)	200 OK + JSON
GET	/api/eficiencia/operarios?estado=En Proceso&limite=3	Filtra por estado y limita resultados (Query params combinados)	200 OK + JSON
GET	/api/eficiencia/operarios/:id	Perfil completo de un operario por ID (Path param)	200 OK + JSON
GET	/api/eficiencia/operarios/999	ID inexistente — operario no encontrado	404 Not Found

2.3 Autenticación — API Key

La API usa API Key como mecanismo de autenticación. La clave se envía en el header HTTP x-api-key en cada petición. Si no se incluye o es incorrecta, el servidor rechaza la petición con un error 401 o 403 antes de ejecutar cualquier consulta a la base de datos.

- API Key en el header HTTP: x-api-key: texticode-2026
- La clave se almacena en el archivo .env del backend como API_KEY_EFICIENCIA
- El middleware apiKey.js verifica la clave en todas las rutas de /api/eficiencia
- Sin la clave correcta → 401 Unauthorized o 403 Forbidden

```
// texticode-api/apiKey.js
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key) return res.status(401).json({ ok: false,
    mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key inválida' })
  next()
}
export default verificarApiKey

// texticode-api/.env
API_KEY_EFICIENCIA=texticode-2026
```

2.4 Parámetros disponibles

Tipo	Parámetro	Valores aceptados	Comportamiento
Path	:id	Número entero (ej: 27, 28)	Devuelve el perfil de ese operario + detalle de sus órdenes. 404 si no existe.
Query	?rendimiento	Alto Medio Bajo	Filtra la lista por nivel de rendimiento. 400 si el valor no es válido.
Query	?estado	Completada En Proceso Pausado	Filtra operarios que tengan órdenes en ese estado.
Query	?limite	Número entero positivo (ej: 3, 5)	Limita el número de resultados devueltos. 400 si no es un número positivo.

3. IMPLEMENTACIÓN TÉCNICA REAL EN TEXTICODE

3.1 Estructura del proyecto con la API implementada



Archivo	Rol en la API de Eficiencia
routes/eficiencia.js	Archivo principal. Define los dos endpoints GET: uno con Path param (/operarios/:id) y otro con Query params (/operarios). Importa db.js y el middleware apiKey.js.

apiKey.js	Middleware de seguridad. Verifica que el header x-api-key coincida con la variable de entorno API_KEY_EFICIENCIA. Rechaza con 401 si falta y 403 si es incorrecta.
.env	Almacena la API Key de forma segura: API_KEY_EFICIENCIA=texticode-2026. Nunca se sube al repositorio gracias a .gitignore.
db.js	Conexión mysql2/promise al servidor MySQL local. La API hace dos queries SQL usando JOIN entre las tablas usuario, rol y orden_produccion.
index.js	Punto de entrada del backend. Registra la ruta: app.use('/api/eficiencia', eficienciaRouter). El endpoint queda disponible en localhost:3001.

3.2 Código del endpoint con Path param

FUNCIONALIDAD 1: Perfil de operario por ID (Path param)

Cuando se consulta `/api/eficiencia/operarios/:id`, el sistema devuelve el perfil completo del operario: nombre, estadísticas de producción y el detalle de todas sus órdenes asignadas ordenadas por fecha límite.

```
// routes/eficiencia.js - Path param
router.get('/operarios/:id', async (req, res) => {
  const { id } = req.params
  if (isNaN(id))
    return res.status(400).json({ ok: false, mensaje: 'El id debe ser un número válido' })

  const [rows] = await db.query(`
    SELECT u.Id_Usuario, u.Nombre_Completo, u.Nombre_Usuario, u.Correo, u.Telefono,
    COUNT(CASE WHEN op.Estado = 'Completada' THEN 1 END) AS ordenes_completadas,
    COUNT(CASE WHEN op.Estado = 'En Proceso' THEN 1 END) AS ordenes_en_proceso,
    COALESCE(SUM(op.Unidades_Realizadas), 0) AS
total_unidades_producidas,
    ROUND(COUNT(CASE WHEN op.Estado = 'Completada' THEN 1 END) * 100.0
    / NULLIF(COUNT(op.Id_Orden), 0), 1) AS prendas_por_dia,
    CASE
      WHEN COUNT(CASE WHEN CURDATE() > op.Fecha_Limite AND op.Estado IN ('En
Proceso', 'Pausado') THEN 1 END) > 0 THEN 'Bajo'
      WHEN ROUND(COUNT(CASE WHEN op.Estado = 'Completada' THEN 1 END) * 100.0 /
NULLIF(COUNT(op.Id_Orden), 0), 1) >= 10 THEN 'Alto'
      WHEN ROUND(COUNT(CASE WHEN op.Estado = 'Completada' THEN 1 END) * 100.0 /
NULLIF(COUNT(op.Id_Orden), 0), 1) >= 5 THEN 'Medio'
      ELSE 'Bajo'
    END AS rendimiento
    FROM usuario u
    INNER JOIN rol r ON u.Id_Rol = r.Id_Rol AND r.Nombre_Rol = 'operario'
    LEFT JOIN orden_produccion op ON op.Id_Operario = u.Id_Usuario
    WHERE u.Id_Usuario = ? AND u.Estado = 'activo'
    GROUP BY u.Id_Usuario, u.Nombre_Completo, u.Nombre_Usuario, u.Correo,
u.Telefono
  `)
```

```

    `, [id])

    if (rows.length === 0)
        return res.status(404).json({ ok: false, mensaje: `No se encontró operario con id ${id}` })

    const [ordenes] = await db.query(
        'SELECT Id_Orden, Producto, Estado, Prioridad, Unidades_Realizadas, Fecha_Limite
        FROM orden_produccion WHERE Id_Operario = ? ORDER BY Fecha_Limite ASC', [id])

    res.json({ ok: true, data: { ...rows[0], ordenes_detalle: ordenes } })
})

```

3.3 Código del endpoint con Query params

FUNCIONALIDAD 2: Lista filtrada de operarios (Query params)

Cuando se consulta `/api/eficiencia/operarios` con parámetros opcionales, el sistema devuelve la lista completa de operarios filtrada y ordenada según los criterios del administrador.

```

// routes/eficiencia.js - Query params
router.get('/operarios', async (req, res) => {
    const { rendimiento, estado, limite } = req.query
    // ... query SQL igual que arriba pero sin WHERE Id_Usuario ...

    let resultado = rows

    // Filtro por rendimiento
    if (rendimiento) {
        const validos = ['Alto', 'Medio', 'Bajo']
        if (!validos.includes(rendimiento))
            return res.status(400).json({ ok: false,
                mensaje: `rendimiento inválido. Usa: ${validos.join(', ')}` })
        resultado = resultado.filter(r => r.rendimiento === rendimiento)
    }

    // Filtro por estado de órdenes
    if (estado) {
        resultado = resultado.filter(r => {
            if (estado === 'Completada') return r.ordenes_completadas > 0
            if (estado === 'En Proceso') return r.ordenes_en_proceso > 0
            if (estado === 'Pausado') return r.ordenes_pausadas > 0
        })
    }

    // Límite de resultados
    if (limite) {
        const n = parseInt(limite)
        if (isNaN(n) || n <= 0)
            return res.status(400).json({ ok: false, mensaje: 'limite debe ser un número positivo' })
        resultado = resultado.slice(0, n)
    }
})

```

```

    }

    res.json({ ok: true, total: resultado.length,
      filtros_aplicados: { rendimiento, estado, limite }, data: resultado })
  })
}

```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

- La API Key se almacena exclusivamente en el archivo .env del backend (texticode-api/.env)
- .env se agrega a .gitignore — nunca se sube al repositorio
- El middleware apiKey.js intercepta toda petición a /api/eficiencia antes de ejecutar cualquier lógica
- Los errores de validación (id inválido, rendimiento incorrecto) se responden con mensajes claros sin exponer detalles internos del servidor

```

# texticode-api/.env — archivo local, NUNCA subir al repositorio
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=texticode
PORT=3001
API_KEY_EFICIENCIA=texticode-2026 ← nunca en el repo

# texticode-api/.gitignore
.env
node_modules/

```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Exposición de la API Key	Uso de .env + .gitignore. La key nunca se expone en el frontend ni en el repositorio.
Acceso sin autenticación	Middleware apiKey.js protege todos los endpoints. Sin key: 401. Key incorrecta: 403.
Inyección SQL	Uso de queries parametrizadas con ? en mysql2. Ningún valor del usuario se concatena directamente al SQL.
ID inválido en Path param	Validación isNaN(id) antes de ejecutar la query. Responde 400 con mensaje claro.
Filtro inválido en Query param	Array de valores válidos con includes(). Rechaza valores desconocidos con 400.

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API en Texticode?

Texticode gestiona órdenes de producción y operarios, pero no tenía forma de medir el rendimiento del equipo. El administrador no sabía cuántas prendas producía cada operario por día, ni cuáles tenían órdenes vencidas sin completar. La API resuelve esto calculando automáticamente métricas de producción desde la base de datos: prendas producidas por día, órdenes en retraso y nivel de rendimiento (Alto / Medio / Bajo), sin intervención manual. La lógica no penaliza al operario por tener tareas en proceso — solo evalúa productividad real y cumplimiento de fechas límite.

P2. Métodos HTTP y datos que retorna

GET /api/eficiencia/operarios: devuelve un array JSON con todos los operarios activos y sus métricas. Incluye: Id_Usuario, Nombre_Completo, prendas_por_dia, total_unidades_producidas, ordenes_en_retraso, ordenes_completadas, ordenes_en_proceso, ordenes_pausadas y rendimiento. GET /api/eficiencia/operarios/:id: devuelve el mismo objeto del operario más el campo ordenes_detalle, un array con cada orden asignada (producto, estado, prioridad, unidades, fecha límite y campo en_retraso).

P3. ¿Cómo se autentica?

Mediante API Key en el header HTTP x-api-key. El middleware apiKey.js lee req.headers['x-api-key'] y lo compara con process.env.API_KEY_EFICIENCIA. Si el header no existe responde 401 Unauthorized. Si existe pero no coincide responde 403 Forbidden. Solo si coincide exactamente pasa al siguiente handler con next(). La clave nunca viaja en la URL ni en el body.

P4. Ventajas de una API interna vs externa

Al ser interna, la API no tiene costo de uso, no depende de disponibilidad de terceros, accede directamente a la base de datos sin latencia de red externa y puede personalizar exactamente los cálculos que necesita el negocio. Una API externa como una de análisis de datos cobraría por consulta y no tendría acceso directo al esquema de Texticode.

P5. Riesgos de seguridad

El principal riesgo es la exposición de la API Key en el repositorio, lo que permitiría a cualquier persona consumir los datos de producción del sistema. Un segundo riesgo es la inyección SQL si los parámetros del usuario se concatenaran directamente en las queries. Un tercer riesgo es la falta de validación de parámetros, que podría generar errores internos del servidor. Los tres se mitigan con .env + .gitignore, queries parametrizadas con mysql2 y validaciones explícitas con mensajes de error controlados.

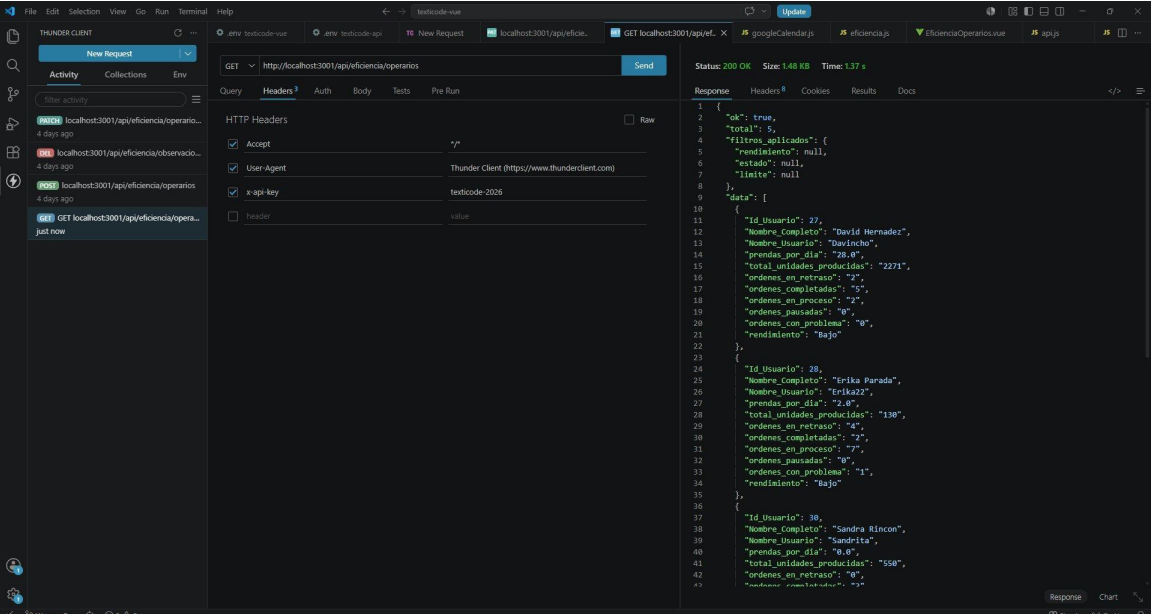
P6. Mejora implementada en Texticode

La API se implementó directamente en el sistema real. El archivo routes/eficiencia.js define los dos endpoints con Path y Query params. El middleware apiKey.js protege todos los accesos. La ruta se registró en index.js igual que el resto de rutas del proyecto. El administrador ahora puede consultar el rendimiento de cualquier operario en tiempo real desde Postman o desde el frontend, con datos calculados automáticamente desde la base de datos de producción.

6. EVIDENCIAS DE PRUEBA

6.1 GET /api/eficiencia/operarios — Lista de operarios con métricas

Se consultó GET /api/eficiencia/operarios con la API Key correcta en el header (x-api-key: texticode-2026). El servidor respondió con Status 200 OK y la lista completa de 5 operarios activos, incluyendo métricas calculadas en tiempo real: prendas por día, total unidades producidas, órdenes en retraso, completadas, en proceso, pausadas y nivel de rendimiento.

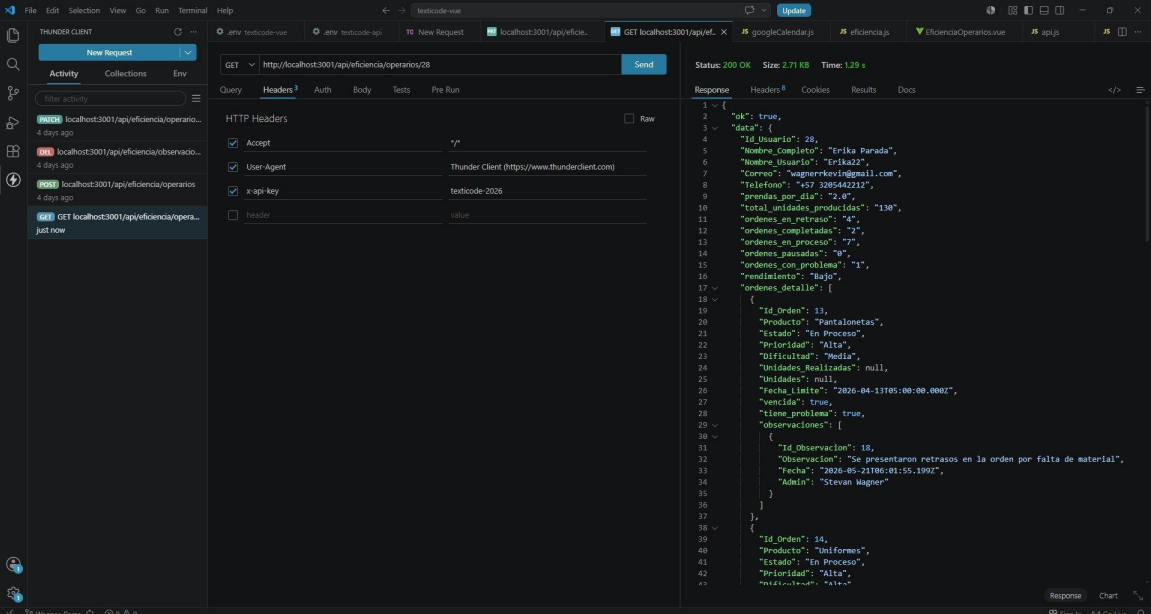


The screenshot shows the Thunder Client interface with a GET request to `http://localhost:3001/api/eficiencia/operarios` successfully executed. The response is a JSON array of 5 operators, each with a unique ID and various performance metrics.

Id Usuario	Nombre Completo	Nombre Usuario	prendas_por_dia	total_unidades_producidas	ordenes_en_retraso	ordenes_completadas	ordenes_en_proceso	ordenes_pausadas	ordenes_con_problema	rendimiento
27	David Hernandez	Davidcho	28.0	2271	2	5	2	0	1	Bajo
28	Erika Parada	Erika22	2.0	130	4	2	7	0	1	Bajo
30	Sandra Rincon	Sandrita	6.0	550	0	0	0	0	0	Bajo

6.2 GET /api/eficiencia/operarios/:id — Perfil completo de operario (ID: 28)

Se consultó GET /api/eficiencia/operarios/28 con la API Key correcta. El sistema devolvió el perfil completo de Erika Parada (@Erika22): 2.0 prendas por día, 130 unidades producidas, 4 órdenes en retraso, rendimiento Bajo. Incluye el detalle de todas sus órdenes asignadas con producto, estado, prioridad, dificultad, fecha límite, campo vencida y las observaciones registradas por orden.



The screenshot displays the Thunder Client interface with a REST client request and its response. The request is a GET to `http://localhost:3001/api/eficiencia/operarios/28` with an `x-api-key` header. The response is a 200 OK status with a JSON body.

Request Details:

- Method: GET
- URL: `http://localhost:3001/api/eficiencia/operarios/28`
- Headers: `x-api-key` (value: `testcode-2026`)

Response Details:

- Status: 200 OK
- Size: 2.71 KB
- Time: 1.29 s

Response Body (JSON):

```
{
  "ok": true,
  "data": {
    "id_usuario": 28,
    "nombre_completo": "Erika Parada",
    "nombre_usuario": "Erika22",
    "correo": "wagnerkevin@gmail.com",
    "telefono": "+57 3285442212",
    "prendas_por_dia": 2.0,
    "total_unidades_producidas": 130,
    "ordenes_en_retraso": 4,
    "ordenes_completadas": 2,
    "ordenes_en_proceso": 7,
    "ordenes_pausadas": 0,
    "ordenes_con_problema": 1,
    "rendimiento": "Bajo",
    "ordenes_detalle": [
      {
        "id_orden": 19,
        "producto": "Pantalones",
        "estado": "En Proceso",
        "prioridad": "Alta",
        "dificultad": "Meda",
        "unidades_realizadas": null,
        "unidades": null,
        "fecha_limite": "2026-04-13T05:00:00.000Z",
        "vencida": true,
        "tiene_problema": true,
        "observaciones": [
          {
            "id_observacion": 18,
            "observacion": "Se presentaron retrasos en la orden por falta de material",
            "fecha": "2026-05-21T06:01:55.199Z",
            "admin": "Steven Wagner"
          }
        ]
      },
      {
        "id_orden": 14,
        "producto": "Uniformes",
        "estado": "En Proceso",
        "prioridad": "Alta",
        "dificultad": "Alta"
      }
    ]
  }
}
```

6.3 GET /api/eficiencia/operarios/:id/historial — Tendencia de rendimiento (ID: 28)

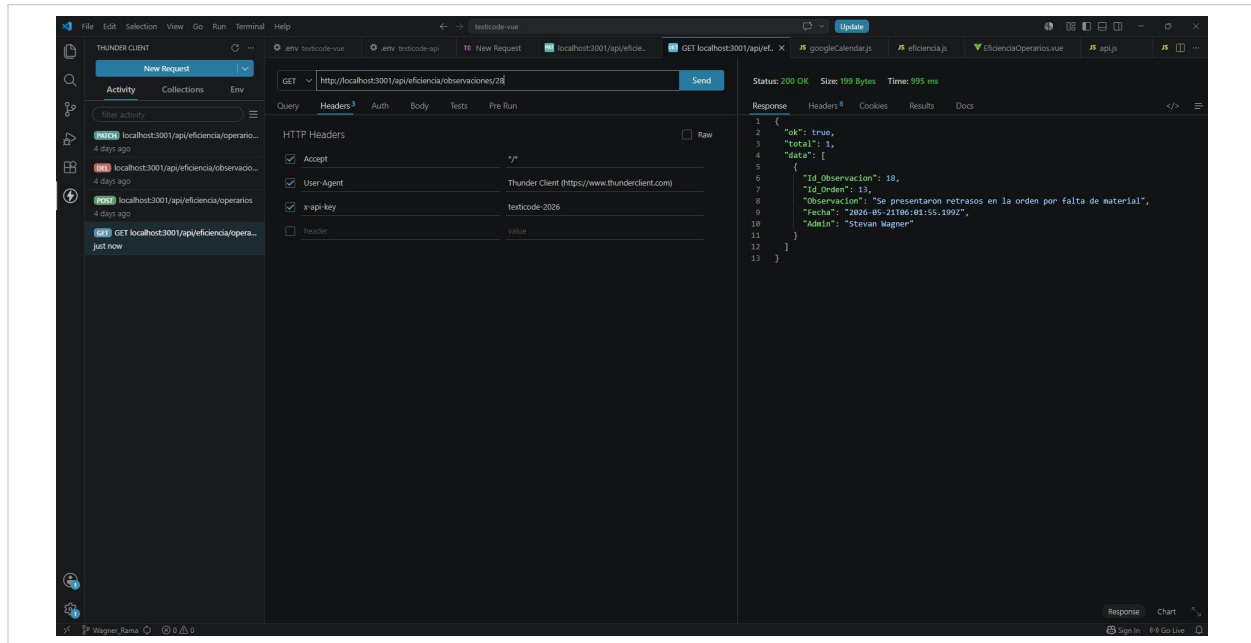
Se consultó GET /api/eficiencia/operarios/28/historial sin parámetro de período (default: semana). El sistema devolvió la comparativa entre el período actual y el anterior: esta semana 0 prendas/día vs semana anterior 7 prendas/día. La tendencia calculada es “bajando” con una diferencia de -7 prendas. El campo rendimiento refleja el nivel correspondiente a cada período.

The screenshot displays the Thunder Client interface with a GET request to `http://localhost:3001/api/eficiencia/operarios/28/historial`. The request headers include `Accept`, `User-Agent`, and `x-api-key`. The response is a JSON object with the following structure:

```
1 {
2   "ok": true,
3   "data": {
4     "operario": {
5       "id_usuario": 28,
6       "nombre_completo": "Erika Parada",
7       "nombre_usuario": "erika22"
8     },
9     "periodo": "semana",
10    "dias": 7,
11    "actual": {
12      "prendas_por_dia": 0,
13      "total_unidades": "0",
14      "completadas": "0",
15      "en_curso": "2",
16      "retrasos": "1",
17      "con_problemas": "0",
18      "rendimiento": "Bajo"
19    },
20    "anterior": {
21      "prendas_por_dia": 7,
22      "total_unidades": "50",
23      "completadas": "1",
24      "retrasos": "0",
25      "rendimiento": "Medio"
26    },
27    "tendencia": "bajando",
28    "diferencia_prendas": -7
29  }
30 }
```

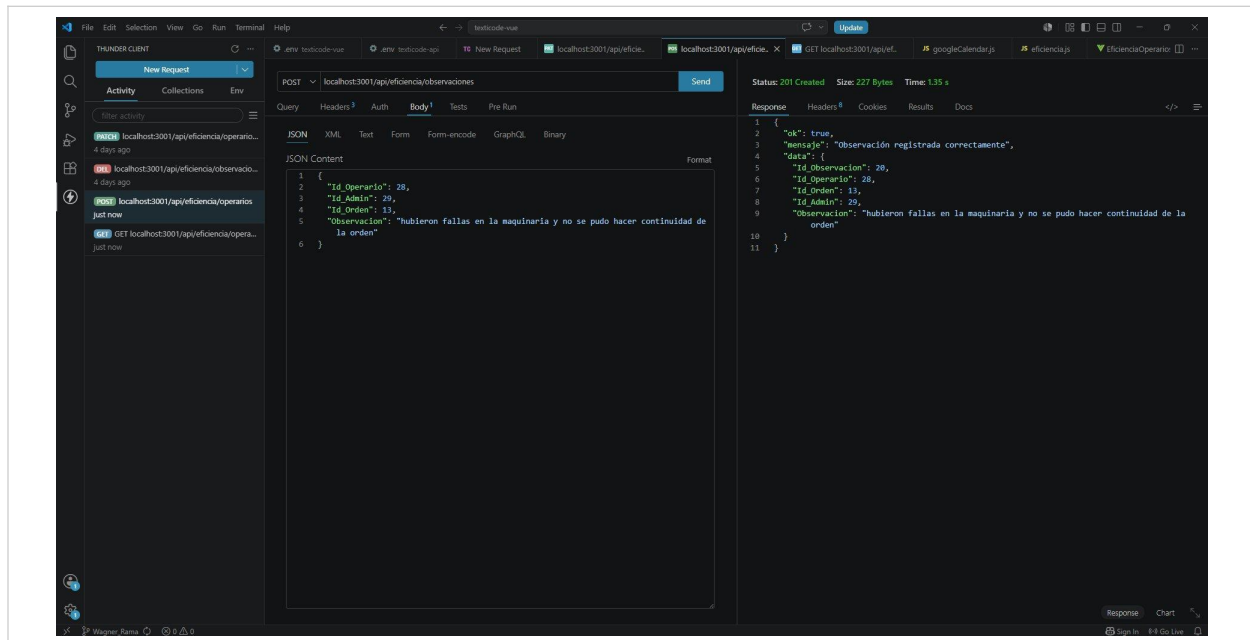
6.4 GET /api/eficiencia/observaciones/:id_operario — Observaciones de un operario (ID: 28)

Se consultó GET /api/eficiencia/observaciones/28 con la API Key correcta. El sistema devolvió 1 observación registrada para el operario: Id_Observacion 18, asociada a la Orden 13, con el texto “Se presentaron retrasos en la orden por falta de material”, fecha 2026-05-21 y el administrador que la registró (Stevan Wagner).



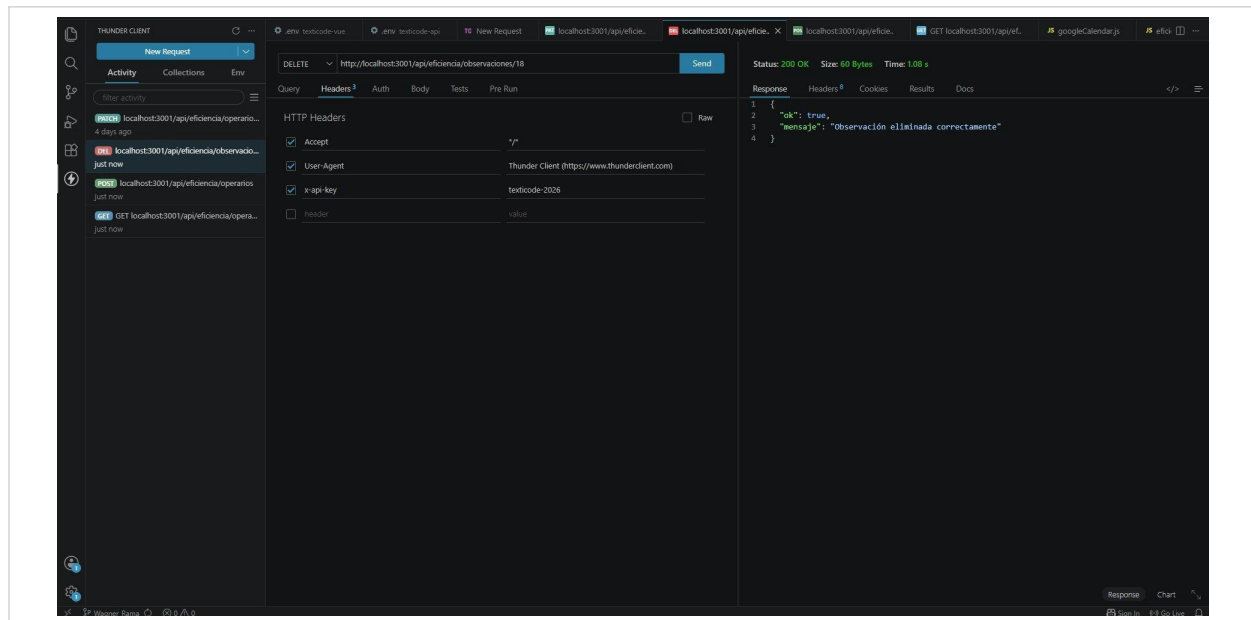
6.5 POST /api/eficiencia/observaciones — Crear observación

Se envió POST /api/eficiencia/observaciones con el body JSON: Id_Operario: 28, Id_Admin: 29, Id_Orden: 13, Observacion: "hubieron fallas en la maquinaria y no se pudo hacer continuidad de la orden". El servidor respondió con Status 201 Created y retornó el objeto creado con Id_Observacion 20, confirmando que la observación quedó registrada correctamente en la base de datos.



6.6 DELETE /api/eficiencia/observaciones/:id — Eliminar observación (ID: 18)

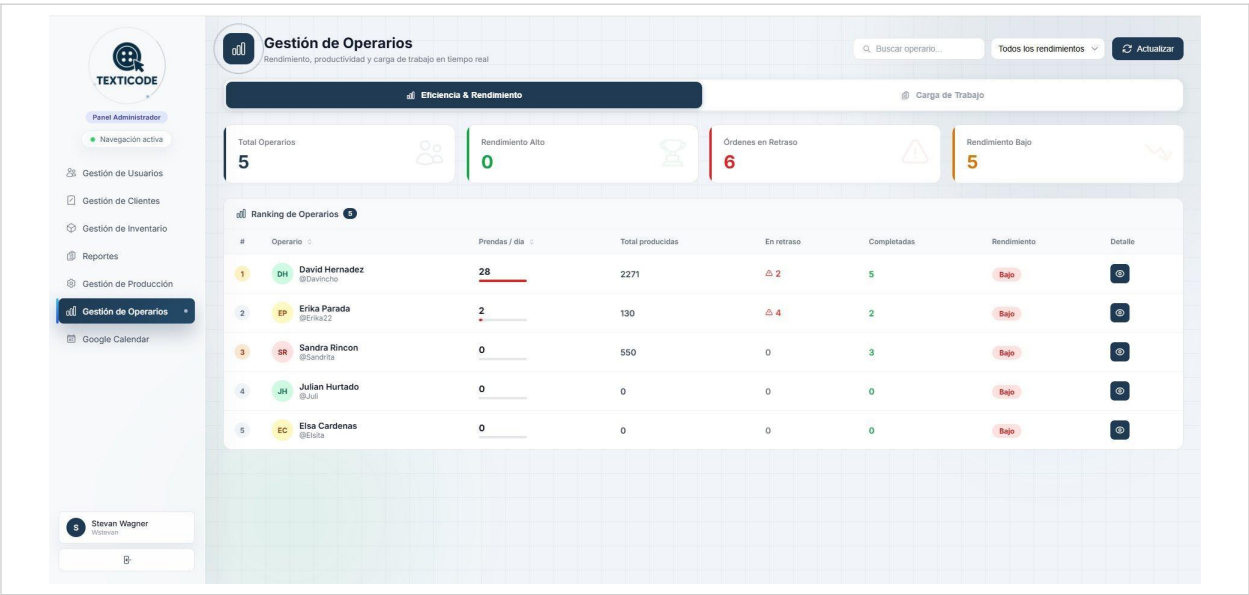
Se envió DELETE /api/eficiencia/observaciones/18 con la API Key correcta. El servidor respondió con Status 200 OK y el mensaje “Observación eliminada correctamente”, confirmando que el registro fue eliminado de la base de datos sin dejar datos huérfanos.



6.7 Frontend — Vista Gestión de Operarios (Ranking)

La vista `EficienciaOperarios.vue` consume el endpoint `GET /api/eficiencia/operarios` y muestra en tiempo real el ranking de los 6 operarios del sistema. Se visualizan las tarjetas de resumen

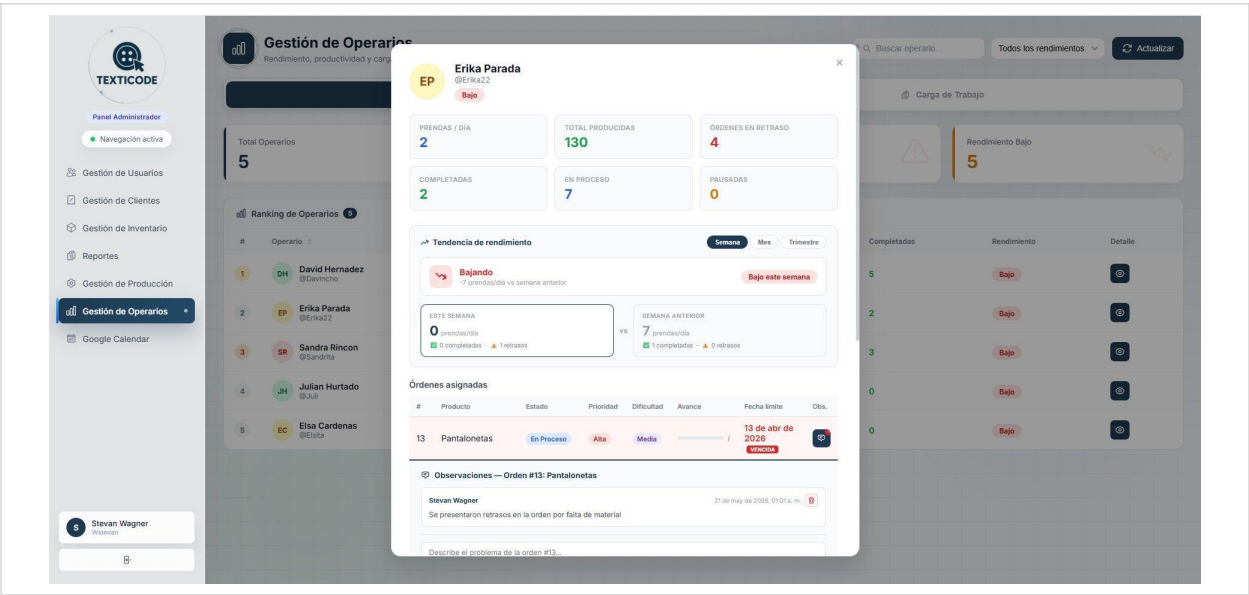
(Total Operarios, Rendimiento Alto, Órdenes en Retraso, Rendimiento Bajo), la tabla de ranking con mini-barras de productividad y los badges de rendimiento (Alto, Medio, Bajo). La navegación lateral del sidebar refleja la ruta /admin/eficiencia activa.



6.8 Frontend — Modal de detalle de operario con historial y observaciones

Modal de detalle del operario Erika Parada (@Erika22). Muestra las estadísticas individuales, la sección de Tendencia de rendimiento con comparativa semana actual vs semana anterior (0 vs 7 prendas/día, tendencia “Bajando”), la tabla de órdenes asignadas con estado, prioridad,

dificultad, fecha límite y estado vencida, y el panel de observaciones con la nota registrada por el administrador y la opción de eliminarla.



7. FLUJO DE INTEGRACIÓN COMPLETO

7.1 Diagrama de flujo — Los 2 endpoints de la API

#	Actor	Acción	Respuesta
1	Admin	Envía GET /api/eficiencia/operarios/28 con x-api-key correcto	200 OK — Perfil completo de Erika Parada + detalle de sus órdenes
2	Middleware	apiKey.js verifica el header x-api-key	Si falta → 401 Si es incorrecta → 403 Si es correcta → next()
3	Controller	Valida que :id sea numérico	Si no es número → 400 Si no existe → 404
4	Base de datos	JOIN entre usuario, rol y orden_produccion con WHERE Id_Usuario = ?	Resultado con métricas calculadas por SQL
5	Admin	Envía GET /api/eficiencia/operarios?rendimiento=Alto	200 OK — Lista filtrada solo con operarios de rendimiento Alto
6	Controller	Aplica filtros: rendimiento, estado, limite sobre el array resultado	Lista reducida según los Query params enviados

7.2 Justificación — API Interna vs Externa

Sobre la elección de API Interna vs Externa

Se descartaron APIs externas de análisis (Google Analytics, Mixpanel) porque Texticode es un sistema cerrado de gestión textil — los datos de producción no deben salir del servidor local. Se descartaron APIs de reporting externas porque requerirían enviar datos sensibles del negocio a terceros. La API interna permite calcular las métricas directamente en la base de datos de Texticode con SQL optimizado, sin costos, sin latencia de red externa y con control total sobre los datos del equipo de producción.